Adam Edwards
CS 414 Programming Language
Dr. Lewis

Assignment 06

In CS 372, I built a binary search tree using a Node struct with raw pointers for the left and right children. Empty branches were represented by nullptr. I also had to manage memory using new, delete, and the Rule of Five. For this assignment, I used std::variant to represent a tree as either Empty or Node, similar to OCaml. I used std::unique_ptr to handle memory cleanup automatically. However, copying the tree would require adding a deep copy function.

The insertion logic is mostly the same. Smaller values go left, larger values go right, and duplicates are ignored. The main difference is how each version handles an empty tree. In CS 372, I checked for nullptr, while this version uses std::visit with lambdas to handle Empty and Node, much like an OCaml match. If the visitor can't handle one of these cases, the program won't compile. With pointers, a missing nullptr check could compile but crash at runtime. I also implemented all three depth-first traversals instead of just inorder. I struggled more with understanding std::visit and how the lambdas worked together, and the pointer approach was easier for me to follow. Overall, I think std::variant makes the tree states clearer and reduces manual memory management, but it's more complicated than doing the same thing in OCaml.